

Module 3: Automating Research Tasks

Claude Code for Academic Researchers

Strategies for Working Within the Context Window

Recall from Module 2: Claude Code can only “see” a limited amount of text at once. Errors compound when it loses track of earlier decisions.

Five strategies to work within this constraint:

- ① Keep tasks focused
- ② Maintain project documentation
- ③ Summarize work between conversations
- ④ Validate iteratively
- ⑤ Write token-aware prompts

What Happens When the Context Window Fills Up

As a conversation grows, Claude Code eventually runs out of room.

When this happens, it **auto-compacts**: it summarizes older parts of the conversation to free up space.

What auto-compacting does:

- Replaces earlier messages with a compressed summary
- Keeps recent messages and tool results intact
- Happens automatically. You will see a notice in the chat

The risk: Details from early in the conversation (variable definitions, decisions you discussed, constraints you mentioned) may be lost or simplified in the summary.

You can also compact manually:

Type `/compact` to force a summary and free up context space.

Useful when you notice Claude Code forgetting earlier decisions or when you want to pivot to a new subtask within the same conversation.

This is why CLAUDE.md matters: it survives compacting because it is reloaded from disk, not

Strategy 1: Keep Tasks Focused

One topic, one conversation.

Avoid:

- “Clean the data, build the panel, run the regressions, and make tables”
- Long conversations that drift across topics
- Asking Claude Code to hold your entire project in its head

Instead:

- “Clean and reshape the raw census file”
- Start a new conversation for each distinct task
- Let each conversation do one thing well

Why

A focused task fits comfortably in the context window. A sprawling task forces Claude Code to forget earlier steps, and that is where silent errors enter.

Strategy 2: Maintain Project Documentation

Claude Code reads certain files **automatically** every time it starts.

Keep these up to date:

- `CLAUDE.md`: conventions, constraints, variable definitions
- `README.md`: project structure, what each directory contains
- Directory-level `CLAUDE.md` files for subfolders with specific rules

Tell Claude Code what to ignore:

- `.claudeignore` works like `.gitignore`
- Exclude large data files, build artifacts, logs
- Prevents Claude Code from wasting context on irrelevant files

The Payoff

Good documentation means Claude Code starts every conversation already knowing your conventions, without you spending tokens re-explaining them.

Strategy 3: Summarize Work Between Conversations

When you finish a conversation, the context is gone. The next conversation starts fresh.

Before ending a conversation:

- 1 Ask Claude Code to summarize what it did and what remains
- 2 Save that summary (in a file, a commit message, or your notes)
- 3 Paste the summary into the *next* conversation as context

Example prompt:

```
Summarize what we did in this session: what files  
were changed, what decisions were made, and what  
still needs to be done.
```

Think of it like a handoff memo between research assistants.

The next “assistant” knows nothing unless you brief it.

Strategy 4: Validate Iteratively

The Compounding Error Problem

If step 1 is wrong and you do not catch it, steps 2–5 build on that mistake. By the time you check the final output, the error is buried.

Check each step before moving to the next:

- ① Clean the data → *verify row counts and distributions*
- ② Construct variables → *spot-check values against known cases*
- ③ Run the merge → *confirm pre- and post-merge row counts*

Slower per step, but **much faster overall** because you never have to unwind a chain of errors you didn't catch.

Strategy 5: Token-Aware Prompting

Every word in your prompt costs tokens. Make them count.

A good prompt tells Claude Code three things:

- ❶ **What files matter:** “Read data/raw/census.csv and scripts/clean.py”
- ❷ **What the goal is:** “Create a county-year panel with one row per county per decade”
- ❸ **What constraints apply:** “Keep all 3,143 counties; do not drop rows with missing population”

Vague:

“Clean up the data and make it ready for analysis”

Specific:

“Read census.csv, drop rows where year < 1950, and save as panel_clean.csv”

The Rule

Be specific about *inputs*, *outputs*, and *constraints*. Be brief about everything else. Context is finite; spend it on what matters.

Validation: Check Known Properties

Before you look at results, ask: *what do I already know must be true?*

Examples:

- A 1:1 merge should not change the number of rows
- A county-year panel with 50 states $\times$ 10 years = 500 rows
- A share variable must be between 0 and 1
- No duplicate county-year pairs after reshaping

Ask Claude Code to assert these explicitly:

```
After the merge, assert that the output has the
same number of rows as the left table and that
there are no duplicate fips-year pairs.
```

If the assertion fails, you catch the error immediately, not three steps later.

Validation: Write Down Expectations First

The Pre-Registration Analogy

Just as you pre-register hypotheses before seeing results, write down what a correct transformation should produce *before* you run it.

Before a merge or transformation, state:

- Expected output dimensions (rows $\times$ columns)
- Which columns should appear and their types
- How missing values should be handled
- A specific row you can trace through by hand

Then compare the actual result to your expectations.

Discrepancies are either a bug or a misunderstanding, both worth catching early.

Validation: Unit Tests Against Known Values

Pick a handful of observations and compute the correct answer by hand.
Then verify the code reproduces them.

Example prompt:

```
Write a test that verifies:  
- Cook County (fips=17031) in 2010 has  
  population = 5,194,675  
- The migration_share for fips=06037  
  in 2000 is between 0.28 and 0.32
```

Why this works:

- Forces you to engage with the data at the observation level
- Catches silent recoding, wrong joins, and off-by-one errors
- Tests survive across refactors. Rerun them after every change

Validation: Replicate Before You Extend

Building on someone else's work?

Have Claude Code reproduce their results first.

- ① Give Claude Code the original paper's code or description
- ② Ask it to replicate a key table or figure
- ③ Compare output to the published result
- ④ Only after replication succeeds, begin your extension

Why

If you cannot reproduce the baseline, anything you build on top of it is uninterpretable. Replication is not a detour; it is the foundation.

Validation: Adversarial Agents

Use a *second* Claude Code conversation as a critic of the first.

How:

- ① Agent 1 writes the code
- ② Open a new conversation (Agent 2) and ask it to review:
“Read `scripts/build_panel.py` and identify any bugs, silent data loss, or decisions that could change the results.”
- ③ Agent 2 has fresh context and no attachment to the code

Why this is powerful:

- The writing agent has blind spots because it wrote the code and “believes” it is correct
- A fresh agent reads the code like a skeptical reviewer
- Catches issues you would miss because you watched it being built

Treat your agents like co-authors who check each other's work, not a single assistant you trust unconditionally.